

```

module Sequential_Binary_Multiplier (Product, Ready, Multiplicand, Multiplier,
Start, clock, reset_b);
// Default configuration: five-bit datapath
parameter dp_width = 5; // Set to width of datapath
output [2*dp_width -1: 0] Product;
output Ready;
input [dp_width -1: 0] Multiplicand, Multiplier;
input Start, clock, reset_b;
parameter BC_size = 3; // Size of bit counter
parameter S_idle = 3'b001, // one-hot code
S_add = 3'b010,
S_shift = 3'b100;

reg [2: 0] state, next_state;
reg [dp_width -1: 0] A, B, Q; // Sized for datapath
reg C;
reg [BC_size -1: 0] P;
reg Load_regs, Decr_P, Add_regs, Shift_regs;
// Miscellaneous combinational logic
assign Product = {A, Q};
wire Zero = (P == 0); // counter is zero
// Zero = ~|P; // alternative
wire Ready = (state == S_idle); // controller status
// control unit
always @ (posedge clock, negedge reset_b)
if (~reset_b) state <= S_idle; else state <= next_state;
always @ (state, Start, Q[0], Zero) begin
next_state = S_idle;
Load_regs = 0;
Decr_P = 0;
Add_regs = 0;
Shift_regs = 0;
case (state)
S_idle: if (Start) begin next_state = S_add; Load_regs = 1; end
S_add: begin next_state = S_shift; Decr_P = 1; if (Q[0]) Add_regs = 1; end
S_shift: begin Shift_regs = 1; if (Zero) next_state = S_idle;
else next_state = S_add; end
default: next_state = S_idle;
endcase
end
// datapath unit
always @ (posedge clock) begin
if (Load_regs) begin
P <= dp_width;
A <= 0;
C <= 0;
B <= Multiplicand;
Q <= Multiplier;
end
if (Add_regs) {C, A} <= A + B;

```

```
    if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;  
    if (Decr_P) P <= P -1;  
end  
endmodule
```
